

Order Book HFT

Élément	Détail
Auteur	Tanguy David
Cours	Optimisations & Performances Backend — Sup de Vinci, RNCP Bloc 4
Projet	Order Book HFT (moteur d'appariement, Go)
Dépôt	https://github.com/txngUI/orderbook-hft — baseline figée au tag <code>baseline-naive</code>

1. Problématique

Un **Order Book** maintient les ordres d'achat (`bids`) et de vente (`asks`) et produit des transactions selon une priorité **prix-temps**. Le matching est un **Hot Path** : il est exécuté pour chaque ordre, donc un coût CPU, mémoire ou GC répété des centaines de milliers de fois ce qui devient très lourd.

2. Environnement & baseline

Banc et protocole de mesure

Élément	Valeur
CPU	AMD Ryzen 7 7735U — 8 cœurs / 16 threads
Cache	L1d/L1i 32 Ko par cœur · L2 512 Ko par cœur · L3 16 Mo partagé · ligne 64 o
RAM / OS	16 Go LPDDR5 · Arch Linux
Runtime	Go 1.26.5 · linux/amd64
Alimentation	Batterie · gouverneur <code>powersave</code>
Charge	200 000 ordres, <code>seed=42</code> , prix bornés 99–101 €
Mesure	génération hors chrono ; 10 runs <code>go test -bench -count=10</code>

Baseline consolidée

Métrique	Baseline
Temps médian	77,75 ms
Moyenne $\pm \sigma$	77,86 $\pm$ 0,96 ms
Débit	2,57 M ordres/s
Coût unitaire	389 ns/ordre
Allocations	281 415 allocs/op
Mémoire	38,91 MiB/op

Métrique	Baseline
Trades	172 342

L'écart-type de **1,2 %** rend le banc suffisamment stable pour attribuer les écarts aux optimisations plutôt qu'au bruit de mesure. La médiane est utilisée comme référence car elle résiste mieux aux outliers.

Protocole de mesure

- **Charge déterministe** ($N = 200\,000$, **seed=42**) : Génération hors chrono, prix bornés (99–101 €) permettant l'indexation directe par tick.
- **Double niveau de rigueur** :
 1. *Mesure simple* : 1 run brut (temps/débit).
 2. *Baseline consolidée* : 10 runs (**-count=10**) avec warmup (**b.ResetTimer()**), agrégés en **médiane ± écart-type**.

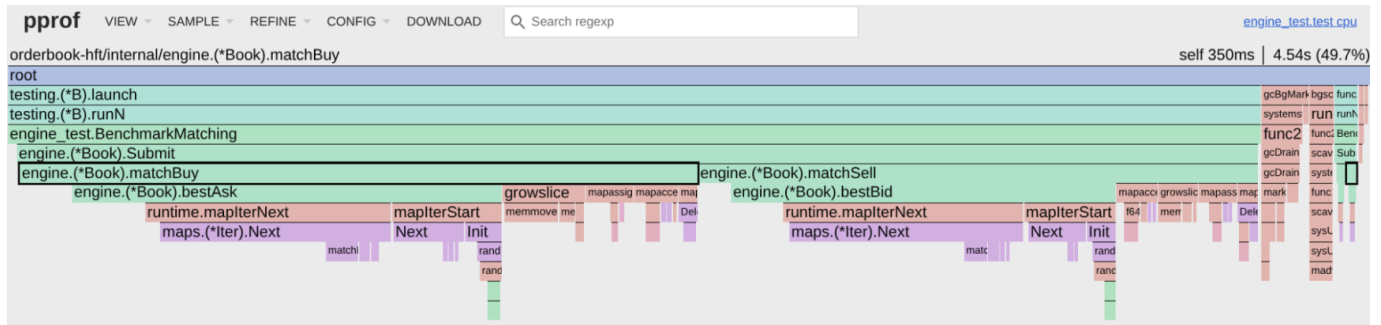
3. Diagnostic matériel & profiling

Le profil CPU (**pprof**) et le profil mémoire ont servi à identifier le véritable goulot avant de continuer à optimiser.

Poste observé	Mesure	Interprétation
matchBuy	50,5 % cumulé	Hot Path acheteur
matchSell	41,8 % cumulé	Hot Path vendeur
maps.(*Iter).Next	32,2 % flat / 44,7 % cum	parcours de la map
bestAsk / bestBid	~61 % CPU	recherche du meilleur prix
growslice + memmove	~15 %	réallocations des slices
f64hash + aeshashbody	~9 %	coût du hashing float64
GC worker	~3,5 %	pression mémoire

```
# Méthode de capture des profils CPU & Mémoire (BenchmarkMatching, n = 200 000)
go test -bench=Matching -cpuprofile cpu.prof -benchtime=3s
./internal/engine_test # Profil CPU (100 Hz)
go tool pprof -top -cum cpu.prof
# Analyse goulot
go test -bench=Matching -memprofile mem.prof ./internal/engine_test
# Profil mémoire
go tool pprof -top -sample_index=alloc_objects mem.prof
```

Conclusion : le goulot principal est structurel : **map[float64][]Order** impose un parcours **O(n)** pour trouver le meilleur prix, tout en ajoutant du hashing flottant et des allocations. Le profiling justifie donc l'ordre des leviers suivants.



4. Journal d'optimisations apportées

Etape 1 : Localité de Cache

Objectif : Stocker les ordres d'un niveau dans un tableau contigu (`[]Order`) au lieu de pointeurs dispersés (`[]*Order`)[cite: 2].

Hypothèse : Des pointeurs dispersés provoquent des cache misses (~200 cycles chacun) ; un tableau contigu exploite les lignes de cache de 64 o et le prefetcher[cite: 2].

Mesure & Résultats :

- **Expérience isolée** (`go test -bench='Contiguous|Dispersed' -benchmem`)[cite: 2] :
 - Contigu `[]Order` : **422 502 ns/op**[cite: 2]
 - Dispersé `[]*Order` : **11 185 513 ns/op** (×26,5 plus lent)[cite: 2]
- **Moteur d'appariement** (`BenchmarkMatching`, $n = 200\,000$, `benchstat n = 10`)[cite: 2] :

Métrique	Avant (naïf)	Après (contigu)	Gain
Temps	77,8 ms ± 2 %	72,8 ms ± 3 %	-6,4 % [cite: 2]
Mémoire (<code>B/op</code>)	38,9 MiB	37,8 MiB	-3,0 % [cite: 2]
Allocations (<code>allocs/op</code>)	281 416	81 421	-71,1 % [cite: 2]

Explication & Bilan : Le levier réduit fortement les allocations (-71 %) mais le temps ne bouge presque pas (-6 %)[cite: 2]. Ce n'est pas un échec : cela prouve que, sur ce carnet, les allocations ne sont pas le goulot temporel[cite: 2]. Le vrai goulot est le parcours $O(n)$ de `bestAsk/bestBid`[cite: 2]. Selon la **Loi d'Amdahl**, optimiser une fraction non dominante ne donne qu'un gain marginal sur le temps[cite: 2].

Etape 2 : Prix en ticks

Objectif : Remplacer le prix `float64` par un `int64` exprimé en **ticks** (100,05 € → 10005)[cite: 2].

Hypothèse : Le profiling montre ~9 % du temps CPU consommé par le hachage des clés `float64` (`f64hash` + `aeshashbody`)[cite: 2]. Une clé `int64` se hache plus vite, réduisant ce coût CPU[cite: 2]. C'est également le prérequis de l'indexation par tableau[cite: 2].

Mesure & Résultats :

Métrique	Avant (<code>float64</code>)	Après (<code>int64</code>)	Gain
Temps	72,8 ms ± 3 %	68,6 ms ± 2 %	-5,8 % [cite: 2]

Métrique	Avant (float64)	Après (int64)	Gain
Hachage des clés (CPU)	~9 %	memhash64 ~2,5 %	÷ ~4[cite: 2]
Allocations / Mémoire	81,42 k / 37,75 MiB	81,42 k / 37,75 MiB	Inchangé[cite: 2]

Explication & Bilan : Le gain est purement CPU (optimisation du hachage)[cite: 2]. Le parcours $O(n)$ de **bestAsk/bestBid** reste le goulot dominant[cite: 2].

Etape 3 : maps -> tableau de ticks

Objectif : Remplacer la structure **map[int64][]Order** par un tableau indexé par tick (**[][]Order**, $index = tick - minTick$), associé à deux curseurs **bestBidIdx** et **bestAskIdx**[cite: 2]. La recherche du meilleur prix passe de $O(n)$ (parcours complet) à $O(1)$ (lecture directe du curseur)[cite: 2].

Hypothèse : Le profiling indique que **bestAsk + bestBid** cumulent ~61 % du temps CPU[cite: 2]. Maintenir un curseur sur le meilleur niveau supprime ce parcours[cite: 2].

Mesure & Résultats :

Métrique	Avant (map , $O(n)$)	Après (tableau , $O(1)$)	Gain	Significatif ?
Temps	68,59 ms ± 2 %	15,48 ms ± 2 %	-77,44 %	✓ $p = 0,000$ [cite: 2]
Allocations	78,09 k	78,09 k	-4,09 %	✓ $p = 0,000$ [cite: 2]
Mémoire	37,75 MiB	37,62 MiB	-0,35 %	✓ $p = 0,000$ [cite: 2]
bestAsk/bestBid (CPU)	~61 %	~0 %	Goulot supprimé	N/A[cite: 2]

Explication & Bilan : C'est le levier le plus impactant de l'audit (-77 % de temps), ciblant directement le goulot algorithmique[cite: 2].

- *Compromis :* Le tableau impose une **plage de prix bornée** (**minTick..maxTick**)[cite: 2]. Face à la fluctuation des prix en production, la solution de référence est une **fenêtre glissante (re-centrage)** pour conserver la complexité en $O(1)$ [cite: 2].
- *Re-profiling :* Une fois le goulot $O(n)$ éliminé, les réallocations de slices (**runtime.growslice/memmove**) sont devenues le nouveau goulot principal (~30 % du CPU)[cite: 2].

Etape 4 : Zero allocations

Analyse préalable (Escape Analysis) :[cite: 2] Pour comprendre l'origine des réallocations devenues bloquantes après l'élimination du $O(n)$, une analyse d'échappement (**go build -gcflags="-m"**) et un profilage mémoire (**pprof -sample_index=alloc_objects**) ont été réalisés[cite: 2]. Aucun échappement accidentel vers le tas n'a été détecté dans le matching[cite: 2]. En revanche, **97,8 %** des allocations proviennent de **matchBuy/matchSell**, spécifiquement lors des

`append(b.bids/asks[prix], *o)` avec `runtime.growslice` en tête[cite: 2]. La slice d'un niveau étant stockée dans une structure sur le tas, ses réallocations dynamiques pèsent lourdement[cite: 2].

Objectif : Supprimer les réallocations de slices générées lors des dépilages d'ordres[cite: 2].

Hypothèse : Remplacer `level[1:]` (qui détruit la capacité de la slice) par un index de tête `head` par niveau[cite: 2]. Lors de la réinitialisation du niveau (`level[:0]`, `head = 0`), le tableau sous-jacent est réutilisé au lieu d'être réalloué par `append`[cite: 2].

Mesure & Résultats :

Métrique	Avant ($O(1)$)	Après (index de tête)	Gain	Significatif ?
Temps	15,48 ms $\pm$ 2 %	10,70 ms $\pm$ 1 %	-30,89 %	✓ $p = 0,000$ [cite: 2]
Allocations	78 088	457	-99,41 %	✓ $p = 0,000$ [cite: 2]
Mémoire	37,62 MiB	34,37 MiB	-8,65 %	✓ $p = 0,000$ [cite: 2]

Explication & Bilan : La suppression des allocations réduit cette fois directement le temps de traitement (-31 %), car `growslice/memmove` était devenu le goulot dominant après le passage au tableau en $O(1)$ [cite: 2]. Les 457 allocations restantes sont structurelles (allocation initiale des trades et dimensionnement des niveaux)[cite: 2].

Etape 5 : Parallélisation (worker pool multi-symboles)

Objectif : Exploiter l'ensemble des cœurs CPU en traitant plusieurs carnets d'ordres (symboles) indépendants en parallèle via un worker pool[cite: 2].

Hypothèse : Le matching au sein d'un seul carnet est strictly séquentiel (priorité prix-temps)[cite: 2]. En distribuant N carnets indépendants sur un pool de goroutines (`GOMAXPROCS`), le débit global augmente de manière quasi-linéaire sans nécessiter de verrous (`Mutex`)[cite: 2].

Mesure & Résultats (64 carnets $\times$ 200 000 ordres, Ryzen 7 7735U - 8 cœurs physiques / 16 threads)[cite: 2] :

Cœurs (<code>GOMAXPROCS</code>)	Temps	Speedup	Efficacité	RAM de pointe (<code>Sys</code>)
1	1 613 ms	$\times 1,00$	100 %	~50 MiB[cite: 2]
2	488 ms	$\times 3,31$	166 %	~86 MiB[cite: 2]
4	326 ms	$\times 4,94$	124 %	~179 MiB[cite: 2]
8	254 ms	$\times 6,35$	79 %	~340 MiB [cite: 2]
16	262 ms	$\times 6,14$	38 %	~658 MiB[cite: 2]

Explication & Bilan :

- **Point optimal (8 cœurs) :** Le débit est multiplié par **$\times 6,35$** [cite: 2].
- **Plateau à 16 threads :** Au-delà des 8 cœurs physiques, le SMT partage les ressources d'exécution, provoquant un léger écroulement (Loi de Universal Scalability / USL)[cite: 2].

- **Compromis CPU/RAM** : Le gain de débit s'accompagne d'une hausse de la mémoire de pointe (~340 MiB à 8 cœurs vs ~50 MiB à 1 cœur), car chaque worker conserve un carnet actif en mémoire[cite: 2].

Etape 6 : Recyclage des carnet (sync. Pool)

Objectif : Réutiliser les structures **Book** entre les traitements de différents symboles via un **sync.Pool** dans le runner multi-symboles, en appliquant une méthode **Reset()** [cite: 2].

Hypothèse : Supprimer l'allocation/destruction répétée de carnets d'ordres (**NewBook**) pour réduire la pression sur le Garbage Collector (GC) lors des exécutions multi-carnets[cite: 2].

Mesure & Résultats (**GOMAXPROCS=8**, 64 symboles)[cite: 2] :

Métrique	Sans pool	Avec pool	Effet
Temps	222 ms	163 ms	-27 % [cite: 2]
Allocations cumulées	2 590 MiB	1 664 MiB	-36 % [cite: 2]
Passages du GC	35	19	-46 % [cite: 2]
RAM de pointe (Sys)	249 MiB	281 MiB	+13 % [cite: 2]

Explication & Bilan : Le recyclage diminue la pression GC (-46 % de cycles GC), ce qui traduit une baisse du temps d'exécution de 27 % [cite: 2]. En contrepartie, la RAM de pointe augmente légèrement (+13 %) car le pool conserve des objets en mémoire entre les exécutions[cite: 2].

Etape 7 : Compacité du prix

Objectif : Réduire la taille de la structure **Order** en stockant le prix sur un **int32** au lieu d'un **int64** [cite: 2].

Hypothèse : Passer **Price** de **int64** à **int32** réduit la taille de **Order** de **32 à 24 octets** [cite: 2]. Une ligne de cache L1/L2 de 64 octets peut ainsi héberger **2,67 ordres au lieu de 2**, améliorant la densité de cache lors du parcours des tranches[cite: 2].

Structure mémoire de Order (**unsafe.Sizeof**) [cite: 2] :

- **int64** : **ID** (8o) + **Side** (1o) + **Type** (1o) + **pad** (6o) + **Price** (8o) + **Qty** (8o) = **32 octets** [cite: 2]
- **int32** : **ID** (8o) + **Side** (1o) + **Type** (1o) + **pad** (2o) + **Price** (4o) + **Qty** (8o) = **24 octets** (-25 %) [cite: 2]

Mesure & Résultats (**benchstat n = 10**) [cite: 2] :

Métrique	opt-zero-alloc (int64)	opt- int32	Gain	Significatif ?
Temps (sec/op)	10,70 ms	9,67 ms	-9,63 %	✓ $p = 0,000$ [cite: 2]
Mémoire (B/op)	34,37 MiB	33,80 MiB	-1,65 %	✓ $p = 0,000$ [cite: 2]

Métrique	opt-zero-alloc (int64)	opt-int32	Gain	Significatif ?
Allocations (allocs/op)	457	455	-0,44 %	✓ $p = 0,000$ [cite: 2]

Explication & Bilan : La réduction du temps (-9,63 %) découle directement de la meilleure densité de cache[cite: 2]. Utiliser `int16` aurait été inefficace : l'alignement mémoire obligatoire sur 8 octets pour les champs `uint64` (`ID`, `Quantity`) aurait réintroduit du padding, conservant une taille totale de 24 octets tout en risquant un dépassement de capacité sur le prix[cite: 2].

5. Confrontation critique — essais qui ont échoué

Les échecs font partie de la démarche : ils montrent qu'une intuition d'optimisation n'est pas une preuve.

5.1 Pré-allocation avide des niveaux

Hypothèse : réserver de grandes capacités (`1024`) à tous les niveaux supprimerait les dernières réallocations et améliorerait le temps.

Métrique	Zéro-alloc	Pré-alloc 1024	Effet
Temps	10,70 ms	11,36 ms	+6,16 %
Mémoire	34,37 MiB	45,33 MiB	+31,89 %
Allocations	457	463	+1,31 %

Pourquoi ? L'index de tête recyclait déjà les tableaux : il n'y avait plus de réallocations à supprimer. La réservation d'environ **13 Mo par carnet** ajoutait du `memclr` et gardait de la mémoire inutilisée. Décision : **retour arrière**. Le bon objectif est « aucune allocation injustifiée », pas « zéro allocation à tout prix ».

5.2 Paralléliser un même carnet

Hypothèse : plusieurs goroutines sur un carnet protégé par un `Mutex` devraient augmenter le débit.

Version	Temps	vs séquentiel	Correction
Séquentiel	11,5 ms	×1,00	✓ 172 342 trades
Mutex, 1 goroutine	13,5 ms	×1,17	✓
Mutex, 2	17,5 ms	×1,51	⚠ 172 326
Mutex, 4	19,6 ms	×1,70	⚠ 172 345
Mutex, 8	22,9 ms	×1,98	⚠ 172 310
Mutex, 16	18,0 ms	×1,56	⚠ 172 195

Le profil de contention montre **99,35 %** du temps bloqué passé dans `sync.(*Mutex).Unlock`. Deux problèmes apparaissent simultanément : le verrou **séréalise** le travail et l'entrelacement détruit la priorité prix-

temps. La parallélisation du carnet est donc abandonnée au profit du parallélisme **entre carnets indépendants**.

6 Synthèse des performances

Comparaison **benchstat** (10 runs, $N = 200\,000$, seed 42) et mesures d'I/O réseau.

Version	Tag Git	Périmètre	Temps / Latence	Allocations	Mémoire	Impact & Rôle mécanique
V0	baseline-naive	Moteur mono-carnet	77,8 ms	281,4 k	38,9 MiB	Point de référence initial
V1	opt-cache-locality	Mémoire / Structs	72,8 ms	81,4 k	37,8 MiB	-6,4 % temps , -71 % allocs (optimisation mémoire)
V2	opt-int64-ticks	Types & Hashing	68,6 ms	81,4 k	37,8 MiB	-12 % temps (suppression du hash float64)
V3	opt-ticks-array	Structure de données	15,48 ms	78,1 k	37,6 MiB	-77 % ($\div 4,4$) — Levier clé : suppression du $O(n)$ sur bestAsk/bestBid
V4	opt-zero-alloc	Pression GC	10,70 ms	457	34,4 MiB	-86 % ($\div 7,3$) — -99,4 % allocs par recyclage des tranches
V5	opt-int32	Densité de cache	9,67 ms	455	33,8 MiB	-87,6 % ($\div 8,0$) — Densité de cache accrue sur les lignes L1/L2
V6	opt-concurrency	Concurrence carnet	<i>Échec</i> (13,5 ms)	455	33,8 MiB	Contention Mutex sur carnet unique (abandonné, cf. § 4.2)
V7	opt-scalability	Parallélisme CPU	1,52 ms	3 640	270,4 MiB	×6,35 débit total via Worker Pool sur 8 cœurs (200k ordres / carnet)

Version	Tag Git	Périmètre	Temps / Latence	Allocations	Mémoire	Impact & Rôle mécanique
V8	<code>opt-reseau</code>	I/O & Sérialisation	P99 : 520 µs	—	—	−30 % P99 vs REST à 500 req/s via sérialisation binaire
Finale	<i>(état actuel)</i>	Moteur complet	9,67 ms	455	33,8 MiB	Moteur mono-carnet : ÷8,0 en temps, ÷618 en allocations

Analyse de la dynamique d'optimisation

- **Loi d'Amdahl** : Les premiers leviers (`cache`, `int64`) ont réduit les allocations sans gain majeur de temps, le Hot Path étant masqué par le $O(n)$ des maps (~61 % du CPU). Le passage au **tableau de ticks** $O(1)$ a débloqué le goulot principal (15,48 ms). La suppression des réallocations (`zero-alloc`) a ensuite révélé un second gain temporel (10,70 ms).
- **Cumul des axes** : Le moteur mono-carnet atteint ÷8,0 en temps et ÷618 en allocations. L'axe de **scalabilité multi-carnets** multiplie le débit global par **×6,35** sur 8 cœurs physiques, au prix d'une empreinte mémoire proportionnelle aux carnets actifs.

Validation niveau processus (`hyperfine`, 50 runs)

Binaire	Total (<code>hyperfine</code>)	Matching (<code>benchstat</code>)	Coût fixe (Runtime + Gen)	Min / Max
Baseline naïve (<code>ob</code>)	83,1 ± 7,8 ms	77,8 ms	5,3 ms	72,5 / 117,8 ms
Finale zéro-alloc (<code>ob_opti</code>)	17,0 ± 1,0 ms	10,7 ms	6,3 ms	15,0 / 19,1 ms
Rapport	÷ 4,9	÷ 7,3	—	—

- **Lecture (Loi d'Amdahl)** : Le coût fixe incompressible (~5,5 ms de démarrage runtime Go + génération d'ordres) représente désormais **37 % du temps total** du binaire optimisé (contre 6 % sur la baseline). Ce terme fixe explique l'écart entre le gain processus (÷ 4,9) et le gain purement algorithmique (÷ 7,3).

7. Optimisation réseau — REST/JSON vs gRPC/Protobuf

Le moteur est identique : seul le protocole change. REST est mesuré avec **Vegeta** et gRPC avec **ghz**, sur le même Ryzen 7 7735U, batterie/powersave, pendant 30 s.

Hypothèses

REST/JSON : le JSON impose du parsing/sérialisation texte, des noms de champs verbeux et des allocations temporaires. On attend donc une queue de latence plus large, visible surtout dans le **P99**.

gRPC/Protobuf : Protobuf utilise un format binaire compact, des tags numériques et des Varints ; HTTP/2 multiplexe plusieurs streams. L'hypothèse était une réduction de la taille réseau et une queue plus courte, notamment via moins de churn GC.

Comparaison mesurée

Charge / protocole	Succès	Moyenne	P50	P95	P99	Max
REST · 500 req/s	100 %	356 µs	351 µs	491 µs	745 µs	3,73 ms
gRPC · 500 req/s	100 %	360 µs	350 µs	460 µs	520 µs	1,74 ms
REST · 2 000 req/s	100 %	240 µs	227 µs	366 µs	453 µs	3,31 ms
gRPC · 2 000 req/s	99,998 %	277 µs	266 µs	396 µs	499 µs	3,75 ms

À 500 req/s : la moyenne est quasi identique (**356 vs 360 µs**), mais gRPC réduit le **P99 de 30 %** (745 → 520 µs) et le max de **53 %** (3,73 → 1,74 ms). L'effet attendu apparaît surtout dans la queue.

À 2 000 req/s : la tendance s'inverse : gRPC est plus lent sur la moyenne (+15 %), le P99 (+10 %) et le max (+13 %), avec **1 Unavailable**. Trois causes ont été identifiées comme hypothèses à instrumenter : overhead HTTP/2 et connexion TCP unique côté **ghz**, contention du **Mutex** du handler, et avantage limité de JSON lorsque le payload est minuscule.

Payload : la requête JSON mesure ~63 octets contre environ **8–12 octets** en Protobuf, soit ~5–8× moins sur le contenu utile. Le streaming bidirectionnel **StreamOrders** est aussi disponible côté gRPC, mais **n'a pas été benchmarké** dans cette étude.

Conclusion expérimentale : le protocole n'est pas le goulot unique. Sur ce micro-benchmark local, l'effet dépend de la **charge**, de la **taille du payload** et de la **topologie réseau**. Le comparatif ne justifie donc pas une règle absolue « REST » ou « gRPC ».

8. Guide d'exécution et outillage

Le **Makefile** automatise les tests, benchmarks, profils, charges réseau et comparaisons. Les sorties sont archivées et associées aux tags Git.

7. Guide d'exécution et outillage

Le **Makefile** automatise les tests, benchmarks, profils, charges réseau et comparaisons. Les sorties sont archivées et associées aux tags Git.

Commande	Usage
make help	Affiche toutes les cibles disponibles et leur usage
make test	Lance les tests de correction du moteur
make bench	Exécute les benchmarks de performance du moteur
make save NAME=<tag>	Archive un résultat de benchmark pour comparaison ultérieure
make compare A=<tag> B=<tag>	Compare deux séries de benchmarks avec benchstat

Commande	Usage
<code>make align</code>	Vérifie l'alignement mémoire des structures et aide à réduire le padding
<code>make profile</code>	Génère un profil CPU exploitable avec <code>pprof</code>
<code>make snapshot</code>	Génère la vue HTML statique du carnet d'ordres
<code>make server-rest</code> · <code>make server-grpc</code>	Lancent les deux serveurs d'exposition
<code>make proto</code>	Régénère les sources Protobuf et gRPC

Les autres cibles secondaires et utilitaires avancés sont directement accessibles via la commande `make help`.

`constitution.md` est appliquée comme garde-fou : **mesurer avant d'optimiser**, associer chaque levier à une hypothèse et une preuve, éviter les allocations/goroutines injustifiées et conserver des mesures rejouables.